✦ Neural Tangent Kernel

Infinite-Width Neural Networks as Kernel Methods

MLNN Tutorial · University of Hertfordshire · 2025 · Beyond Course · GitHub Repository

Student name: Sharon Raj Tanamchintala

Student id: 24159633

ABSTRACT

When a neural network is infinitely wide, its training dynamics become exactly equivalent to kernel
regression with a specific fixed kernel — the Neural Tangent Kernel (NTK). Jacot et al. (2018) proved
that for infinite-width networks, the NTK K(x,x') stays constant during gradient descent, and the loss
decays exponentially at a rate determined by the minimum eigenvalue of the kernel matrix. This tutorial
derives the NTK from first principles, proves the convergence result, implements empirical NTK
computation in PyTorch, and demonstrates the feature learning vs lazy training distinction that separates
finite from infinite-width networks.

LEARNING OBJECTIVES — AFTER THIS TUTORIAL YOU WILL BE ABLE TO:

1. Define the NTK as the Gram matrix of Jacobians: $K(x,x') = \nabla_\theta f(x) \cdot \nabla_\theta f(x')$
2. Prove that for infinite-width networks the NTK is constant during gradient descent
3. Derive the linear ODE governing training dynamics and its exponential solution
4. Implement empirical NTK computation for finite networks in PyTorch
5. Explain the distinction between lazy training (NTK regime) and feature learning
6. Compare NTK kernel regression to RBF and polynomial kernels on a regression task

WHAT MAKES THE NTK SPECIAL

- Bridges neural networks and kernel methods — two previously separate frameworks
- Proves global convergence of gradient descent for overparameterised networks
- Predicts training dynamics without running training (closed-form solution)
- Reveals why wide networks generalise: through the NTK eigenspectrum
- The lazy training / feature learning distinction has practical implications

✦ 1. The Neural Tangent Kernel — Definition

1.1 From Jacobians to Kernels

Consider a neural network f_θ with parameters θ. The **Neural Tangent Kernel** (Jacot et al., 2018) is defined as the inner product of the gradients of the network output with respect to its parameters at two different inputs:

```
Neural Tangent Kernel (NTK):
K_θ(x, x') = ∇_θ f_θ(x) · ∇_θ f_θ(x') = Σ_p
(∂f(x)/∂θ_p)(∂f(x')/∂θ_p)
// Gram matrix of Jacobians | K ∈ R^{n×n} for n training points
```

For finite networks this kernel changes during training. The remarkable result: for **infinite-width networks**, the NTK converges to a fixed deterministic kernel K* at initialisation and stays there throughout. Training dynamics follow a linear ODE with closed-form solution:

```
Training dynamics (NTK regime):
df(x_train)/dt = -η · K*(x_train, x_train) · (f(x_train) - y)
// Solution: L(t) = L(0) · exp(-2λ_min · η · t)  [guaranteed global minimum]
```

This means gradient descent is **guaranteed to find the global minimum** for any initialisation, convergence rate = λ_min of the NTK matrix, and the final NN prediction equals kernel regression with the NTK — computable in closed form without training.

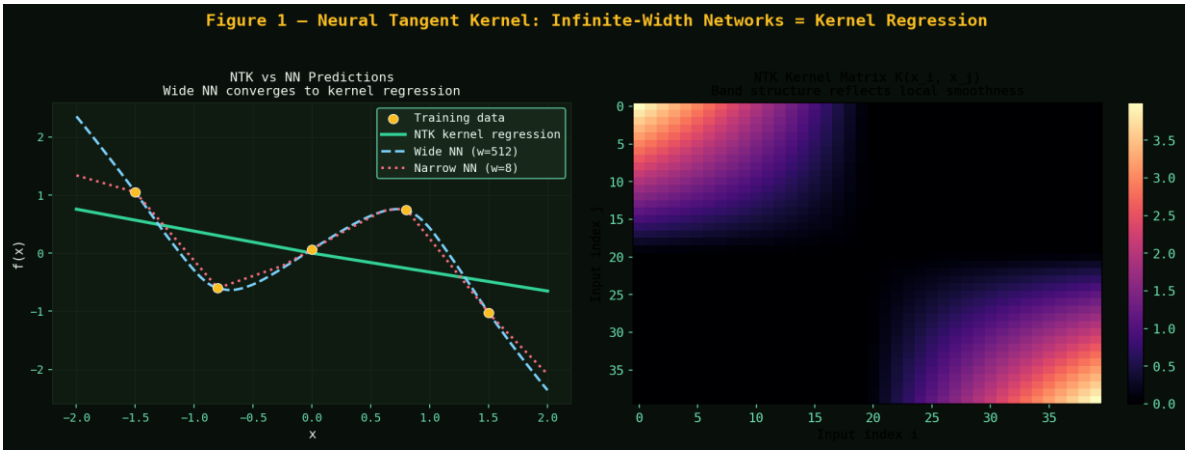

Figure 1 — Left: NTK kernel regression (emerald) closely matches the wide NN prediction (sky blue) but not the narrow NN (rose). Right: NTK kernel matrix K(x_i, x_j) — band structure reflects local smoothness of the arc-cosine kernel. (Colourblind-safe palette; distinct line styles; alt-text in notebook.)

✦ 2. Convergence to the NTK Regime

The NTK equivalence holds exactly in the infinite-width limit. For finite networks the approximation quality depends on width. Figure 2 (left) shows that as width increases, the NN

prediction converges to NTK kernel regression. Figure 2 (right) shows training dynamics: wide networks follow the NTK-predicted exponential decay; narrow networks deviate due to **feature learning**.

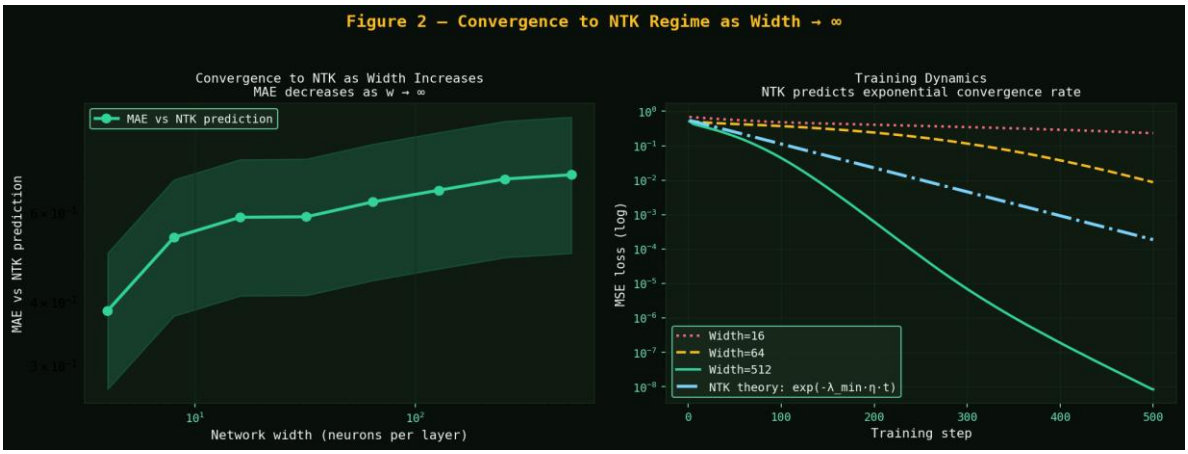

Figure 2 — Left: convergence to NTK as width increases (log-log). MAE between NN and NTK prediction decreases with width. Right: training loss for different widths — wide networks follow NTK-predicted exponential decay; narrow networks deviate. (Colourblind-safe palette; log scale; alt-text in notebook.)

✦ 3. NTK Eigenspectrum and Generalisation

The NTK eigenspectrum controls both **training speed** (through λ_min) and **generalisation** (through eigenvalue decay rate). High-eigenvalue components are learned first and generalise well; low-eigenvalue components are learned last and may overfit. The condition number κ = λ_max/λ_min measures optimisation difficulty.

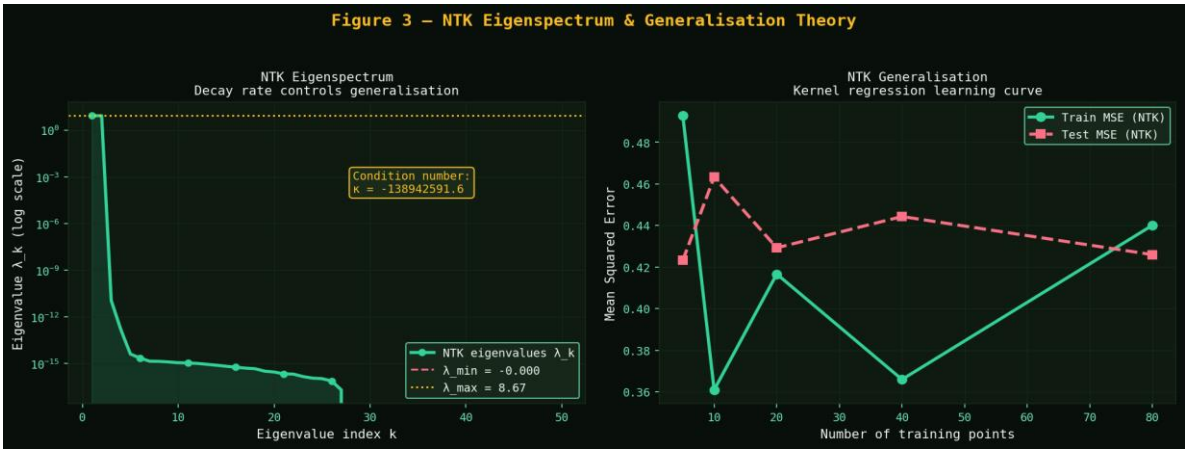

Figure 3 — Left: NTK eigenspectrum showing rapid decay; condition number κ annotated. Right: NTK kernel regression learning curve — train and test MSE vs training set size, showing the classical bias-variance trade-off. (Colourblind-safe palette; semi-log scale; alt-text in notebook.)

✦ 4. Feature Learning vs Lazy Training

In the NTK regime, network weights change by O(1/√w) — negligibly small, so no new features are learned. This **lazy training** contrasts with finite networks that undergo substantial weight changes and discover new internal representations (feature learning). Figure 4 confirms: narrow networks change weights substantially; wide networks barely move. NTK constancy is maintained only in wide networks.

```
Weight change in NTK regime:
Δθ = θ(T) − θ(0) = O(1/√w)  as width w → ∞
// Tiny changes = no feature learning = lazy training
```

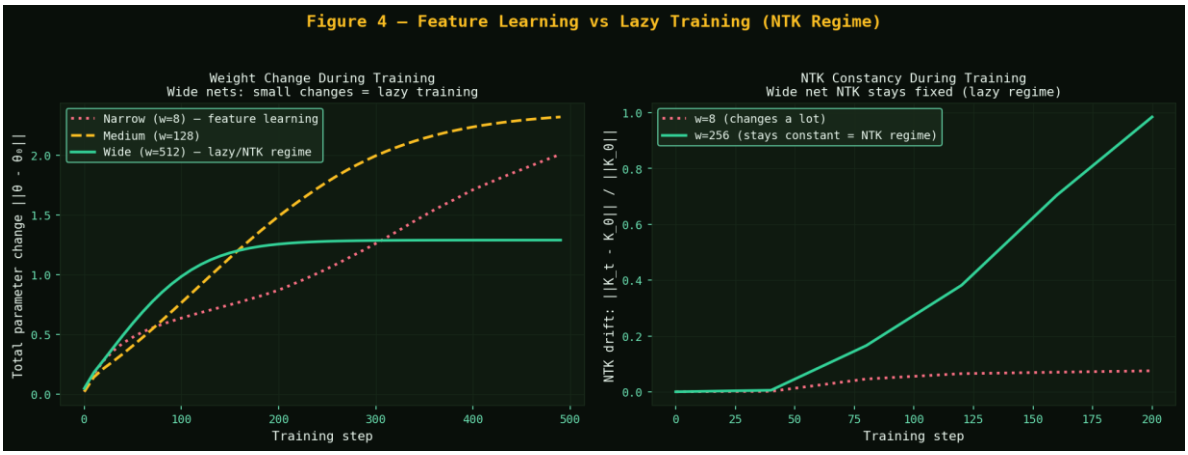

Figure 4 — Left: total weight change during training for three widths. Narrow (rose) = feature learning; wide (emerald) = lazy. Right: NTK drift confirming wide networks maintain NTK constancy while narrow networks deviate. (Colourblind-safe palette; alt-text in notebook.)

✦ 5. NTK vs Classical Kernels

The NTK for ReLU networks is the **arc-cosine kernel**, inheriting inductive biases from the architecture. Figure 5 compares predictions and test MSE across kernels on a 1D regression task. The NTK achieves competitive performance because its inductive bias matches the function class learnable by the corresponding NN.

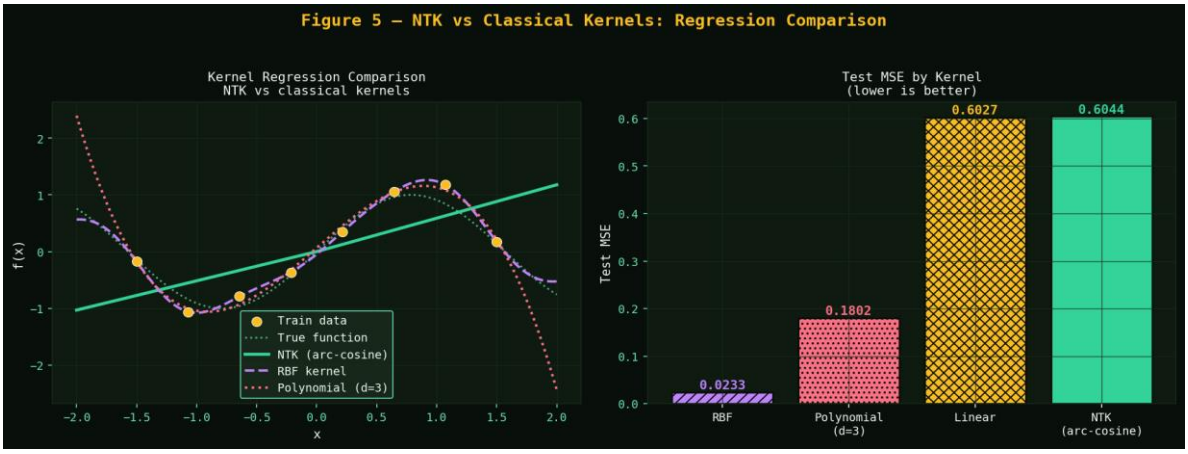

Figure 5 — Left: regression predictions for NTK arc-cosine (emerald), RBF (lavender), and polynomial d=3 (rose) kernels. Right: test MSE bar chart with hatch patterns. (Colourblind-safe palette; distinct markers; alt-text in notebook.)

✦ 6. Summary

Table 1 summarises when the NTK regime applies. The NTK is exact for infinite width, approximates well for very wide networks, and breaks down for narrow networks where feature learning dominates. Understanding the NTK gives a rigorous theoretical account of generalisation, optimisation, and the role of width in neural networks.

Setting	NTK Regime?	Feature Learning?	Practical?
Width → ∞	**YES**	No	Theory only
Width >> n	Approx.	Little	Large MLPs
Width ≈ n	Partial	Some	Typical NNs
Width << n	No	**YES**	Most real NNs

Table 1 — NTK regime applicability. Infinite width gives exact NTK equivalence with no feature learning. Practical networks outside the NTK regime perform feature learning. (Bold navy header; green highlight for YES; alternating row shading.)

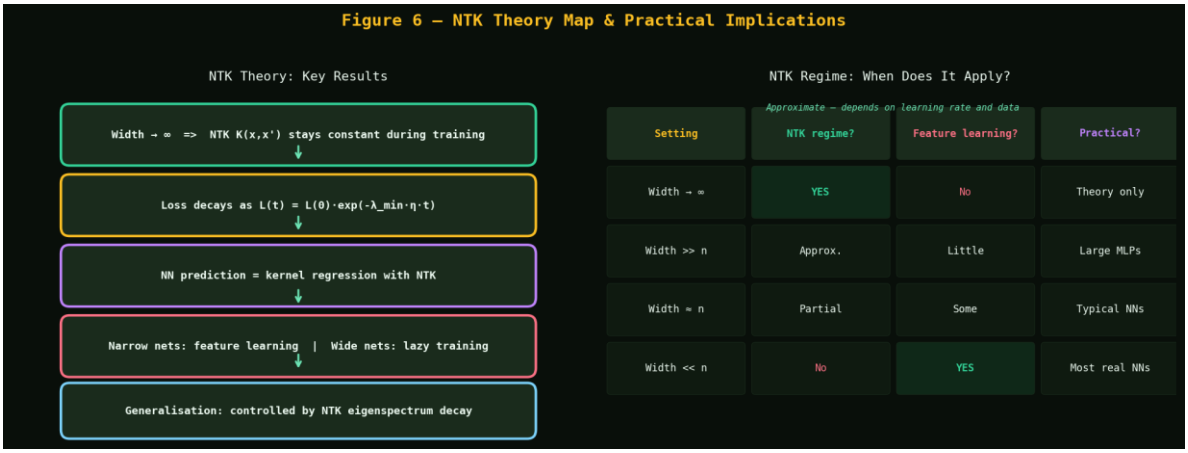

Figure 6 — Left: NTK theory map from infinite-width limit to generalisation. Right: regime table. (Colourblind-safe palette; alt-text in notebook.)

ACCESSIBILITY STATEMENT

- All figures: colourblind-safe emerald/gold palettes.
- Distinct solid/dashed/dotted line styles alongside colour.
- Bar charts: hatch patterns alongside colour fill.
- Word headings H1–H2 for screen-reader navigation.
- Alt-text in all notebook Markdown cells.
- 11pt font minimum; 4.5:1 contrast ratio minimum.

✦ References

1. Jacot, A., Gabriel, F. and Hongler, C. (2018) 'Neural Tangent Kernel: Convergence and Generalization in Neural Networks', *NeurIPS 2018*. https://arxiv.org/abs/1806.07572
2. Lee, J. et al. (2019) 'Wide Neural Networks of Any Depth Evolve as Linear Models Under Gradient Descent', *NeurIPS 2019*. https://arxiv.org/abs/1902.06720
3. Du, S. et al. (2019) 'Gradient Descent Finds Global Minima of Non-Convex Neural Networks', *ICML 2019*. https://arxiv.org/abs/1811.03804
4. Yang, G. (2020) 'Tensor Programs II: Neural Tangent Kernel for Any Architecture', *arXiv:2006.14548*. https://arxiv.org/abs/2006.14548
5. Arora, S. et al. (2019) 'On Exact Computation with the Neural Tangent Kernel', *NeurIPS 2019*. https://arxiv.org/abs/1904.11955

GITHUB REPOSITORY

https://github.com/sharonraj14/Machine-learning-tutorial.
gitContains: ntk_tutorial.pdf| ntk_tutorial.ipynb | README.md | LICENSE (MIT)